Guía paso a paso — Clase 3: Red neuronal simple (parte 1)

Temas: neurona artificial moderna · qué es una derivada · funciones de activación · descenso por gradiente en una variable · práctica en Python
Duración: ~2 h 30 min
Material: diapositivas `Clase3_Diapositivas` (PDF y PowerPoint, 20 diapositivas) · repositorio https://github.com/datahuba/curso-redes-neuronales
Requisito: Clases 1 y 2 (ambiente `nni` instalado y repositorio clonado)
Lecturas base (libro del Dr. Irving Vasquez):

- Neurona artificial moderna
- Descenso por gradiente (preliminares)

> El módulo 3 del curso ("Red neuronal simple") es largo, así que se divide en dos clases:
>
> - **Clase 3 (esta):** la neurona moderna, las funciones de activación y el descenso por gradiente con una sola variable.
> - **Clase 4:** usar el descenso por gradiente para entrenar los pesos de una neurona.

Objetivos de la clase

Al terminar, cada alumno podrá:

1. Explicar en qué se diferencia la neurona moderna del perceptrón: su activación es **continua y derivable**.
2. Calcular a mano las funciones de activación lineal, sigmoide, tanh y ReLU.
3. Explicar con sus palabras qué es una derivada y para qué sirve.
4. Aplicar a mano varias iteraciones del descenso por gradiente y reconocer el efecto de la tasa de aprendizaje.
5. Programar una neurona con sigmoide y el descenso por gradiente en Python.

La guía tiene cuatro partes:

- **Parte A:** preparar las computadoras antes de clase.
- **Parte B:** teoría explicada desde cero, con ejemplos resueltos.
- **Parte C:** práctica en la computadora, paso a paso.
- **Parte D:** problemas frecuentes.

Agenda

Tiempo	Tema	Diapositivas
0:00	Presentación, objetivos y repaso de la Clase 2 (10 min)	1–4
0:10	B1. Del escalón a la neurona moderna (15 min)	5–6

Tiempo	Tema	Diapositivas
0:25	B2. ¿Qué es una derivada? (10 min)	7
0:35	B3. Funciones de activación (20 min)	8–10
0:55	B4. Ejemplo a mano (5 min)	11
1:00	C1. Práctica: neurona con sigmoide (20 min)	12
1:20	Descanso (10 min)	13
1:30	B5. Descenso por gradiente (25 min)	14–17
1:55	Ejercicios en papel (10 min)	18
2:05	C2. Práctica: descenso por gradiente (20 min)	19
2:25	Cierre, quiz y tarea (5 min)	20

PARTE A — Antes de clase (15 min)

A.1 Actualizar el repositorio

En **Anaconda PowerShell Prompt**:

```
cd ~\projects\curso-redes-neuronales
```

```
git pull
```

Verificación: debe decir `Already up to date.` o listar archivos nuevos, entre ellos `docs/practica_03.py`.

> Si `git pull` se queja de "cambios locales" en un notebook, es porque lo modificaste en clase. Guarda una copia con otro nombre (por ejemplo `1_01_mio.ipynb`) y ejecuta `git checkout -- notebooks` antes de repetir `git pull`.

A.2 Activar el ambiente y probar la práctica

```
conda activate nni
```

```
python docs/practica_03.py
```

Verificación: imprime una tabla de funciones de activación, varias filas de descenso por gradiente y al final `Todas las comprobaciones pasaron.`

Opcional, para ver las gráficas:

```
python docs/practica_03.py --graficar
```

Se abren dos ventanas: las cuatro funciones de activación y los pasos del descenso sobre una parábola. Ciérralas para terminar.

A.3 Checklist

- [ ] `python docs/practica_03.py` funciona.
- [ ] VS Code abre `notebooks/1_04_red_neuronal_simple.ipynb` con el kernel **nni**.
- [ ] Existe la carpeta `notebooks/output` (la usa el notebook `1_05` para guardar una gráfica; viene en el repositorio).
- [ ] En el pizarrón: una parábola grande $y = x^2$ con los puntos $x = 5, 4, 3.2$. La usarás para el descenso por gradiente.
- [ ] Una calculadora a mano (o la del celular) con la función e^x, para el ejemplo de la sigmoide.

PARTE B — Teoría desde cero

Repaso de la Clase 2 (5 min) — diapositivas 1–4

Pregunta a la clase:

- "¿Cómo calcula su salida un perceptrón?" → suma ponderada $h = \Sigma\, w_i x_i + b$, y luego el escalón: 1 si $h \geq 0$, 0 si no.
- "¿Cómo aprende?" → corrige los pesos con el error: $W \leftarrow W + \eta(y - \hat{y})X$.
- "¿Qué no pudo aprender?" → XOR, porque no es linealmente separable.

Hoy damos el siguiente paso: cambiar el escalón por algo "suave" y aprender una herramienta nueva para entrenar, el **descenso por gradiente**.

B1. Del escalón a la neurona moderna (15 min) — diapositivas 5–6

¿Qué tiene de malo el escalón?

Dibuja el escalón en el pizarrón: vale 0 a la izquierda de $h = 0$ y salta a 1 a la derecha.

1. **Es "todo o nada".** Si $h = -0.01$ o $h = -100$, la salida es la misma (0). La neurona no nos dice si estuvo **cerca** o **lejos** de acertar.
2. **Es plano en casi todas partes.** Si movemos un poquito un peso, la salida casi nunca cambia. No hay una "pista" de hacia dónde mover los pesos.
3. **Salta de golpe en $h = 0$.** En ese punto no hay una pendiente definida.

Analogía: buscar las llaves a oscuras con alguien que solo dice "sí las encontraste" o "no las encontraste". Es mucho más fácil si te dice "frío… tibio… caliente": eso es lo que da una función suave.

La neurona artificial moderna

- **1960.** Bernard Widrow y Marcian Hoff proponen **ADALINE** (*ADAptive LInear NEuron*).
- Los **mismos dos pasos** del perceptrón:
    1. combinación lineal: $h = \Sigma\, w_i x_i + b$;

2. activación: $\hat{y} = f(h)$.

- **La diferencia:** f es **continua y derivable**. No salta; cambia poco a poco.
- Se entrena con la **regla delta**, que viene del **descenso por gradiente** (lo vemos hoy y la próxima clase).

En los diagramas modernos, la suma y la activación se dibujan juntas en un solo círculo.

B2. ¿Qué es una derivada? (10 min) — diapositiva 7

No hace falta saber cálculo para seguir el curso; basta esta idea:

> **La derivada de una función en un punto es la pendiente de la curva en ese punto:** cuánto sube (o baja) la función si avanzas un poquito a la derecha.

Dibuja la parábola $f(x) = x^2$ y una recta que solo **toca** la curva en un punto (la **recta tangente**):

- En $x = 3$ la curva sube fuerte hacia la derecha → la pendiente es **positiva y grande**.
- En $x = -3$ la curva baja hacia la derecha → la pendiente es **negativa**.
- En $x = 0$ (el fondo) la curva está plana → la pendiente es **0**.

Para $f(x) = x^2$ la derivada es **$f'(x) = 2x$**:

x	−3	0	1	3
$f'(x) = 2x$	−6	0	2	6

Las dos ideas que usaremos:

1. El **signo** de la derivada dice hacia dónde sube la función.
2. Su **tamaño** dice qué tan empinada está.

Notación: $f'(x)$ se lee "f prima de x". También se escribe df/dx.

B3. Funciones de activación (20 min) — diapositivas 8–10

Una **función de activación** transforma la combinación lineal h en la salida $\hat{y}$. Hoy vemos cuatro:

Función	Fórmula	Rango de salida	En palabras
Lineal	$f(h) = h$	cualquier número	Deja pasar h tal cual. Sirve para **regresión** (predecir números).
Sigmoide	$f(h) = 1 / (1 + e^{-h})$	entre 0 y 1	Un escalón "suave". Sirve para **probabilidades**.
Tangente hiperbólica (tanh)	$f(h) = (e^{h} - e^{-h}) / (e^{h} + e^{-h})$	entre −1 y 1	Como la sigmoide, pero centrada en 0.
ReLU	$f(h) = \max(0, h)$	0 o positivo	Corta los negativos a 0. Es la más usada en redes profundas.

> **e** es un número especial ≈ 2.718 (como π ≈ 3.14). e^{-h} significa "e elevado a menos h".

Tabla de valores (diapositiva 9)

Calcula 2 o 3 casillas en el pizarrón con calculadora, y muestra el resto:

h	lineal	sigmoide	tanh	ReLU
−3	−3	0.0474	−0.9951	0
−1	−1	0.2689	−0.7616	0
0	0	0.5	0	0
1	1	0.7311	0.7616	1
3	3	0.9526	0.9951	3

Que noten:

- la sigmoide **nunca** llega a 0 ni a 1;
- vale exactamente **0.5 en h = 0**;
- tanh es como la sigmoide, pero "estirada" entre −1 y 1.

La sigmoide en detalle (diapositiva 10)

- **Saturación:** para h muy grande o muy negativo, la curva se aplana. Ahí la pendiente es casi 0 y la neurona aprende muy lento. Es un problema conocido.
- **Su derivada tiene una forma muy cómoda:**

$$\sigma'(h) = \sigma(h)\,\big(1 - \sigma(h)\big)$$

h	−3	−1	0	1	3
σ′(h)	0.0452	0.1966	**0.25**	0.1966	0.0452

La pendiente máxima es 0.25, en h = 0, y casi se anula en los extremos (saturación). La usaremos en la Clase 4.

- **Una nota importante:** si todas las neuronas fueran **lineales**, por muchas que pongamos en cadena, el resultado seguiría siendo una recta. Las funciones **no lineales** (sigmoide, tanh, ReLU) son las que permiten aprender patrones curvos.

B4. Ejemplo a mano (5 min) — diapositiva 11

Es el mismo ejemplo que verificarán en el notebook `1_04`.

Datos: entradas X = [0.7, −0.3], pesos W = [0.1, 0.8], sesgo b = −0.1.

1. **Combinación lineal:** h = 0.1·0.7 + 0.8·(−0.3) − 0.1 = 0.07 − 0.24 − 0.1 = **−0.27**
2. **Sigmoide:** $e^{0.27}$ ≈ 1.3100, así que σ(−0.27) = 1 / (1 + 1.3100) = 1 / 2.3100 ≈ **0.4329**
3. **Con tanh** en lugar de sigmoide: tanh(−0.27) ≈ **−0.2636**

Compara con la Clase 2: con el escalón, la salida sería simplemente 0 (porque h < 0). La sigmoide dice algo más rico: "0.43, casi en la frontera".

B5. Descenso por gradiente (25 min) — diapositivas 14–17

El problema (diapositiva 14)

Queremos encontrar el valor de x que hace **lo más pequeña posible** una función f(x). En redes neuronales, f será **el error de la red**, y x serán **los pesos**.

Analogía de la montaña a oscuras: estás en una montaña, de noche, sin linterna. Quieres bajar al valle. No ves nada, pero **sientes con los pies hacia dónde baja el terreno**. Das un paso en esa dirección, vuelves a sentir y repites. Eso es el descenso por gradiente: la "inclinación que sientes" es la **derivada**.

La regla (diapositiva 15)

$$x_{nuevo} = x - \eta\, f'(x)$$

Símbolo	Significa
x	El valor actual (dónde estás parado)
f′(x)	La pendiente en ese punto (hacia dónde sube)
η (eta)	La **tasa de aprendizaje**: el tamaño del paso

¿Por qué el signo menos? La derivada apunta hacia donde la función **sube**. Queremos **bajar**, así que vamos en la dirección **contraria**:

- si la pendiente es positiva (sube a la derecha), restamos → vamos a la izquierda;
- si es negativa (sube a la izquierda), restar un negativo suma → vamos a la derecha.

El algoritmo:

1. Empieza en un valor inicial x_0 (puede ser al azar).
2. Calcula la pendiente f′(x).
3. Actualiza: x ← x − η·f′(x).
4. Repite hasta que x casi no cambie, o un número fijo de veces (**iteraciones**).

Ejemplo resuelto (diapositiva 16)

f(x) = x^2, f′(x) = 2x, η = 0.1.

Un paso desde x = −2:
x_nuevo = −2 − 0.1·(2·(−2)) = −2 − 0.1·(−4) = −2 + 0.4 = **−1.6**.
Comprobamos que mejoró: f(−2) = 4 y f(−1.6) = 2.56 ✓.

Cinco pasos desde x = 5 (hazlos en el pizarrón):

Iteración	x	f(x) = x^2	f′(x) = 2x	x − 0.1·f′(x)
1	5	25	10	**4**
2	4	16	8	**3.2**
3	3.2	10.24	6.4	**2.56**
4	2.56	6.55	5.12	**2.048**

Iteración	x	$f(x) = x^2$	$f'(x) = 2x$	$x - 0.1 \cdot f'(x)$
5	2.048	4.19	4.096	**1.6384**

Los pasos se hacen **más cortos** al acercarse al fondo, porque la pendiente es más pequeña ahí. Nadie le dice al algoritmo que frene: lo hace solo.

La tasa de aprendizaje (diapositiva 17)

Mismo problema ($f(x) = x^2$, desde $x = 5$), distintos valores de η, 5 pasos:

η	Valores de x	Qué pasa
0.01	5 → 4.9 → 4.80 → 4.71 → 4.61 → 4.52	**Muy lento**: pasos diminutos
0.1	5 → 4 → 3.2 → 2.56 → 2.05 → 1.64	**Bien**: baja de forma estable
0.5	5 → 0 → 0 → …	Llega de un salto (solo pasa en este problema tan simple)
0.9	5 → −4 → 3.2 → −2.56 → 2.05 → …	**Oscila**: salta de un lado al otro del valle
1.1	5 → −6 → 7.2 → −8.64 → 10.37 → …	**Diverge**: cada salto lo aleja más

Mensaje clave: η es un **hiperparámetro**: lo elegimos nosotros, no lo aprende la red. Si es muy pequeño, el entrenamiento tarda; si es muy grande, se descontrola.

Ejercicios en papel (10 min) — diapositiva 18

1. Con $f(x) = x^2$, $x_0 = 3$ y $\eta = 0.25$, calcula dos pasos del descenso.
2. Del libro: minimiza $f(x) = 2x + x^2$, empezando en $x_0 = 10$, con $\eta = 0.2$. Calcula 6 valores. (Pista: $f'(x) = 2 + 2x$.)
3. ¿Cuánto vale la sigmoide en $h = 0$? ¿Y su derivada?

Respuestas:

1. $f'(3) = 6 \rightarrow x_1 = 3 - 0.25 \cdot 6 =$ **1.5**; $f'(1.5) = 3 \rightarrow x_2 = 1.5 - 0.75 =$ **0.75**.
2. 10 → **5.6** → **2.96** → **1.376** → **0.4256** → **−0.1446** → **−0.4868**. Se acerca a $x = -1$, que es donde $f'(x) = 2 + 2x$ vale 0 (el mínimo).
3. $\sigma(0) =$ **0.5** y $\sigma'(0) = 0.5 \cdot (1 - 0.5) =$ **0.25**.

PARTE C — Práctica en la computadora

Preparación (todos):

1. Abre VS Code → **File** → **Open Folder** → `C:\Users\TuUsuario\projects\curso-redes-neuronales`.
2. En cada notebook, revisa que el kernel sea **nni** (arriba a la derecha).
3. Ejecuta las celdas **en orden**, de arriba abajo, con **Shift + Enter**. Las partes a completar dicen `# TODO`.

C1. Notebook `notebooks/1_04_red_neuronal_simple.ipynb` (20 min, 3 puntos) — diapositiva 12

Qué hay que hacer:

1. **TODO (1 pt):** completar `sigmoide(h)`.
   - Pista: la fórmula es $1 / (1 + e^{-h})$. En NumPy, e^x se escribe `np.exp(x)`.
2. **TODO (1 pt):** completar la clase `Neurona`:
   - `combinacion_lineal(X)` debe devolver $h = W \cdot X + b$. Pista: `np.dot(self.W, X)` hace la suma de productos.
   - `forward(X)` debe calcular h con el método anterior y luego aplicar `self.activacion(h)`.
3. Ejecuta la celda de prueba.
   Verificación: debe imprimir `Output: 0.4329` (el ejemplo de B4).
4. **TODO (1 pt):** escribir una función `tanh(h)` con la fórmula $(e^h - e^{-h})/(e^h + e^{-h})$, crear una neurona con ella y comparar con la sigmoide.
   Verificación: con los mismos datos, la tanh da ≈ **−0.2636**. Compara también con `np.tanh(-0.27)`.

Errores típicos:

- Escribir `np.exp(h)` en vez de `np.exp(-h)` (falta el signo menos). Da 0.5671 en lugar de 0.4329.
- Olvidar `self.` dentro de la clase (`W` en lugar de `self.W`).

C2. Notebook `notebooks/1_05_descenso_por_gradiente_1D.ipynb` (15 min, 2 puntos) — diapositiva 19

Qué hay que hacer:

1. **TODO (1 pt):** completar `gradient_descent_update(theta, learning_rate)`.
   - Pista: es exactamente la regla de B5, y la derivada ya está programada como `df(x)`.
2. Ejecuta la celda de prueba.
   Verificación: el historial debe ser `[5, 4.0, 3.2, 2.56, 2.048, 1.6384]` (puede mostrar más decimales).
3. Ejecuta la celda de la gráfica: debe dibujar la parábola con los puntos rojos bajando.
4. **TODO (1 pt):** reescribe el ciclo para que se detenga solo cuando el error sea menor que 0.001 (`1e-3`), y cuenta las iteraciones.
   - Pista: usa `while` en lugar de `for`, y un contador.
   - **Verificación:** depende del criterio que elijan, y ambos son válidos si lo explican:
     - si paran cuando el **cambio** $|x_{nuevo} - x| < 0.001$, salen **32** iteraciones;
     - si paran cuando **f(x)** < 0.001, salen **23**.

C3. Demo del docente: `docs/practica_03.py` (5 min)

Proyecta y ejecuta (con `(nni)` activo, dentro del repo):

```
python docs/practica_03.py --graficar
```

Muestra:

- la tabla de activaciones (la misma de B3);
- las cinco tasas de aprendizaje de B5: lento, bien, de un salto, oscila, diverge;
- la gráfica de las cuatro activaciones.

Pregunta a la clase: "¿Qué η usarían?"

Cierre (5 min) — diapositiva 20

Resumen en tres líneas:

1. La neurona moderna cambia el escalón por una función **suave y derivable** (sigmoide, tanh, ReLU…).
2. La derivada es la **pendiente**: dice hacia dónde sube una función.
3. El descenso por gradiente baja paso a paso: $\mathbf{x \leftarrow x - \eta \cdot f'(x)}$, con η ni muy grande ni muy pequeña.

Avance: "La próxima clase usaremos esto mismo, pero en vez de bajar por x^2, bajaremos por **el error de la red**, moviendo **los pesos**."

Quiz de salida (en papel, 3 min):

Pregunta	Respuesta
¿Qué cambia la neurona moderna respecto al perceptrón?	La activación es continua y derivable
¿Entre qué valores está la salida de la sigmoide? ¿Y la de tanh?	0 y 1; −1 y 1
¿Qué representa la derivada en un punto?	La pendiente de la curva ahí
¿Por qué la regla del descenso lleva un signo menos?	Para ir en contra de la pendiente, hacia abajo
¿Qué pasa si la tasa de aprendizaje es muy grande?	Oscila o diverge

Tarea:

1. Terminar los notebooks `1_04` y `1_05` (5 puntos en total).
2. Ejecutar `python docs/practica_03.py --graficar` y cambiar en el archivo el x_0 inicial a −8: ¿llega igual al mínimo?
3. Leer del libro: Descenso por gradiente (aprendizaje), preparación de la Clase 4.

Rúbrica:

Notebook	Puntos
1_04 Red neuronal simple	3
1_05 Descenso por gradiente 1D	2

PARTE D — Problemas frecuentes

Síntoma	Causa	Solución
`NameError: name 'np' is not defined`	No se ejecutó la celda de `import`	Ejecuta las celdas desde la primera, en orden
`TypeError: unsupported format string passed to NoneType`	La función todavía devuelve `None`	Completa el `TODO` y vuelve a ejecutar esa celda **y** la de prueba
La sigmoide da 0.5671 en vez de 0.4329	Falta el signo menos: `np.exp(h)`	Debe ser `np.exp(-h)`
`FileNotFoundError: ... output/gradient_descent_1D.png`	No existe la carpeta `output`	Haz `git pull` (ya viene en el repo) o créala dentro de `notebooks`
`RuntimeWarning: overflow in exp`	h muy grande en `np.exp`	Es solo un aviso; con valores normales no aparece
El ciclo `while` no termina	El criterio de parada nunca se cumple, o η ≥ 1 hace que diverja	Revisa la condición y usa η = 0.1; detén la celda con el botón ■
Las gráficas de `practica_03.py` no aparecen	Falta `--graficar`	`python docs/practica_03.py --graficar`
`git pull` dice que hay cambios locales	Modificaste notebooks del repo	Guarda tu copia con otro nombre y ejecuta `git checkout -- notebooks`